

DataExpert.io Community Academy - Week 1 Grader Feedback

Student

Name: Ceasar Jackson

Final Grade

A

Overall assessment

Thanks for the clear, well-organized submission. Overall, you've implemented a clean dimensional model and both full-backfill and incremental SCD Type 2 logic correctly. The intent of each query is well documented, your use of composite/enum types is thoughtful, and your incremental SCD handling (extend unchanged intervals, split on change, start intervals for new actors) is solid.

What you did well

- Clear grain definitions and comments across all files.
- Cumulative actors table correctly models "one row per actor per year" and appends films while preserving history.
- Sensible `quality_class` recalculation policy: only when there are new films; otherwise retain prior classification.
- `FULL OUTER JOIN` in Query 2 ensures carried-forward actors and brand-new actors are included.
- Type 2 SCD table DDL is fit-for-purpose, with a simple integer time axis and a `CHECK` on date bounds.
- Full backfill (Query 4) leverages `LAG` and a streak-identifier pattern correctly to collapse contiguous states.
- Incremental SCD (Query 5) is well-structured: copy historicals, extend unchanged current intervals, split on change, and add new actor intervals.
- Use of a composite type to `UNNEST` paired rows for changed records is elegant and efficient.

Suggestions and potential improvements

Schema and data types

Consider making fields `NOT NULL` where business rules allow:

- `actors.actor`
- `actors.is_active`
- `actors.quality_class`, if unknown classification should never be allowed.
- `actors.films` with a default empty array such as `DEFAULT '{}'::film_struct[]`.

rating as REAL can introduce precision quirks. Prefer `DOUBLE PRECISION` or `NUMERIC(3,1)` and ensure `AVG(rating)` follows the desired precision policy.

If actor names can change, consider a separate static `actors_dim` keyed by `actorid`, or a separate SCD for actor names.

Query 2 - cumulative actors

Year parameters

Hardcoding 2020/2021 is acceptable for an example, but parameterizing previous/current years with a params CTE or runtime variables would make the scripts reusable.

Films array ordering and deduplication

The incoming year's `ARRAY_AGG` is ordered by `filmid`, but the cumulative result after concatenation is not rebuilt with a guaranteed order. If ordering matters, rebuild from `UNNEST(ly.films || ty.films)` with `ORDER BY filmid`, optionally deduplicating by `filmid`.

Thresholds

The query uses strict `>` cutoffs. If business rules require inclusivity (for example, 8.0 should be star), use `>=` as appropriate.

Re-runs

A plain `INSERT` will fail on a second run for the same year because of the primary key. That may be intentional. If idempotence is desired, establish an explicit policy such as deleting/truncating the target year or using `ON CONFLICT`.

Query 3 - SCD DDL

The primary key includes `current_year`, which cleanly separates SCD snapshots across runs. More advanced schemas could add protection against overlapping intervals within a snapshot, but the submitted approach is appropriate for this assignment.

Potential indexes:

```
actors_history_scd(actorid, current_year)
```

```
actors_history_scd(actorid, start_date, end_date)
```

Query 4 - full backfill

The `did_change` logic works. The first row per actor could be made explicitly responsible for starting a new streak, such as with `ROW_NUMBER() = 1 OR did_change`. The existing `IS DISTINCT FROM` approach should already identify the first row as a change in typical data, but explicit logic can make the intent clearer.

The `latest_year` cross join is a good way to stamp the snapshot.

Query 5 - incremental SCD

The logic is correct and clean:

- `historical_scd` retains closed intervals from the prior snapshot.
- `unchanged_records` extends open intervals.
- `changed_records` splits intervals.
- `new_records` starts intervals for brand-new or reappearing actors.

This depends on the prior-year SCD being open as of 2020 (`end_date = 2020`), which is a good pattern. As with Query 2, parameterizing previous/current years would improve reuse.

Re-run behavior

Re-inserting the same snapshot can cause conflicts. Decide explicitly whether repeat execution should:

- clear the snapshot year before rebuilding it; or
- use `INSERT ... ON CONFLICT` to upsert.

Edge cases to consider

Missing years

If an actor has gaps in the `actors` table, full backfill treats the gap as continuous. This is fine if cumulative snapshots generate a row every year. Otherwise, document whether gaps should preserve continuity or break streaks.

Null ratings

`AVG` ignores nulls. If all incoming ratings are null, `AVG` returns null and the current CASE can fall through to bad. If prior classification should instead be retained, add an explicit null policy.

Performance and scalability

Useful source indexes include:

```
actor_films(year, actorid)
actors(actorid, current_year)
```

Information the grader would need if assumptions differ

1. Exact `quality_class` threshold rules, including strict versus inclusive boundaries.
2. DDL for `actor_films` and a small sample dataset to validate array ordering/deduplication.
3. Whether re-runs should be idempotent, and whether the preferred policy is truncate/delete, upsert, or fail.
4. Whether cumulative actors snapshots exist for every year or can contain gaps.
5. Whether film arrays should be unique by `filmid` and consistently ordered across years.

Final grader assessment

This is a strong submission that shows clear understanding of cumulative dimensional modeling and SCD Type 2 handling. The suggestions are primarily robustness improvements: parameterization, idempotence, array ordering/deduplication, and additional constraints/indexes.

Final Grade: A

Post-grade engineering status

The Week 1 repository also contains post-grade hardening work addressing several of these robustness areas while preserving the original graded state under the `week01-graded-a` Git tag.